

Progressing Domain Condition for Size-Change Termination and the Global Trace Condition with Injective Relations

Stefano Berardi

May, 2025

1 Introduction

2 Size-change termination

Size-change termination (SCT), initially proposed by Lee et al. [1], is a technique for proving the termination of recursive functions. SCT leverages Fermat's principle of infinite descent: to prove a proposition P , assume its negation and derive a contradiction by constructing an infinite descending chain within a well-ordered set. In the context of recursive functions, if the function's variables are interpreted within a well-ordered domain, non-termination can be assumed, and a contradiction derived by demonstrating an impossible infinite descending sequence of variable values. These sequences are formed by analyzing the relationships between variables across recursive calls. Each relationship is abstracted as a *size-change graph*, which represents variable transformations during program execution. To make this concept precise, we begin by presenting some preliminary definitions, following the exposition in [1, 2].

Definition 2.1. (Function names and parameters, [1, 2]): For a program P , let F be a finite set of *function names* that appear in P , such that for each $f \in F$, we have n distinct symbols $x_1^f, \dots, x_n^f$ named the *parameters* of f , where the *arity* of f is defined as $\text{arity}(f) = n$. We define $\text{Par}(f) = \{x_1^f, \dots, x_n^f\}$ as the set of parameters of f .

Definition 2.2. (Size-change graphs, [1, 2]): Let P be a program and F the set of function names, and $f, g \in F$. A size-change graph from f to g , written $G : f \rightarrow g$, is a bipartite graph defined as $G = (\text{Par}(f), \text{Par}(g), E)$, where $E \subseteq \text{Par}(f) \times \{\downarrow, \Downarrow\} \times \text{Par}(g)$ and E does not contain both $x_i^f \xrightarrow{\downarrow} x_j^g$ and $x_i^f \xrightarrow{\Downarrow} x_j^g$.

The edges $x_i^f \xrightarrow{\gamma} x_j^g$ represent the relationship between the values of the variables initially passed to the function, and the values passed in each recursive call. When $\gamma = \downarrow$, we call it a *progressing* edge and it specifies that the value of x_i^f is strictly less than the value of x_j^g . When $\gamma = \Downarrow$, we call it a *weakly-progressing* edge, and it specifies that the value of x_i^f is less than or equal to the value of x_j^g . To illustrate this concept, consider the following example of the Ackermann function, taken from [3]:

Example 2.1. For the program in Figure 1, we set $F = \{\text{ack}\}$, with $\text{arity}(\text{ack}) = 2$. For each recursive call, labeled with 1, 2 and 3, we can respectively define the size-change graphs G_1 , G_2 and G_3 . Note that even though the recursive calls 1 and 2 have different input values, their size-change graphs are exactly the same.

The termination behavior of a program P can be inferred by examining the size-change graphs it induces. In particular, we study how these graphs compose to form *multipaths*, as they reveals whether infinite descending chains (indicatives of termination) can occur.

Definition 2.3. (Composable graphs, [1, 2]): Let $f, g, g', h \in F$, with $G_1 = (\text{Par}(f), \text{Par}(g), E)$ and $G_2 = (\text{Par}(g'), \text{Par}(h), E')$ two size-change graphs. We say that G_1 and G_2 are *composable*, if $g = g'$. We write $G_1 G_2 = (\text{Par}(f) \cup \text{Par}(g) \cup \text{Par}(h), E \cup E')$ for the directed graph obtained by the union of G_1 and G_2 .

```

ack(m,n) = if m=0 then n+1 else
            if n=0 then 1ack(m-1,1) else
            2ack(m-1, 3ack(m,n-1))

```

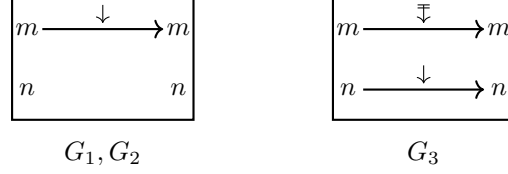


Figure 1: Definition of the Ackermann function a and size-change graphs G_1 , G_2 and G_3

Definition 2.4. (Multipaths, threads and infinite descent[1, 2]) Let $\mathbb{G}$ be a set of size-change graphs:

1. A $\mathbb{G}$ -*multipath* is a finite or infinite sequence $\mathcal{M} = G_1 G_2 \dots$ of size-change graphs in $\mathbb{G}$, such that for every i , G_i and G_{i+1} are composable. $\mathcal{M}$ can be viewed as a (possibly infinite) directed graph.
2. A *thread* in $\mathcal{M}$ is a (possibly infinite) directed path in $\mathcal{M}$.
3. We say that a thread is *progressing* if it contains a progressing edge. Moreover, a thread is *infinitely progressing* if it contains infinitely many progressing edges.
4. A multipath $\mathcal{M}$ has infinite descent if some thread in $\mathcal{M}$ is infinitely descending.

Example 2.2. Suppose there is a hypothetical infinite execution starting from an initial call of the program `ack`, introduced in Example 2.1. This execution is represented by the infinite sequence of recursive calls $c = 1313\dots$ or more compactly $(13)^\omega$. It induces the corresponding multipath $\mathcal{M} = G_1 G_3 G_1 G_3 \dots$ as shown in Figure 2. Within this multipath, we can identify an infinitely progressing thread over the variable m , given by the sequence $m \xrightarrow{\downarrow} m \xrightarrow{\Downarrow} m \xrightarrow{\downarrow} m \dots$

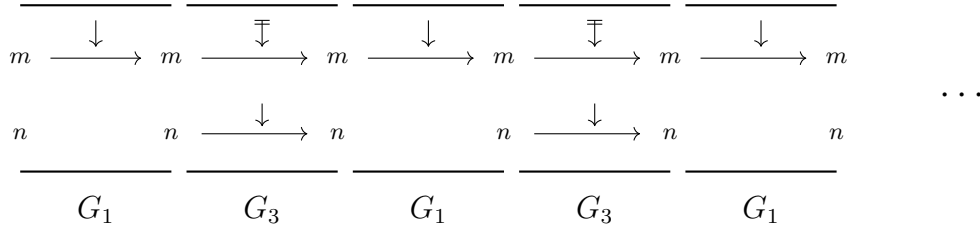


Figure 2: Multipath generated by $(13)^\omega$

The underlying idea of SCT is that, by appealing to the principle of infinite descent, we can prove that a program P terminates if every possible infinite execution would necessarily result in an infinite descending chain of variable values within a well-ordered domain. In other words, given a program P and its associated set of size-change graphs $\mathbb{G}$, we observe that every infinite sequence of recursive calls c in P , induces a corresponding multipath $\mathcal{M}$. If every such multipath exhibits infinite descent, then P cannot have an infinite execution and must terminate. This observation leads to the formal definition of size-change termination condition:

Definition 2.5. (Size-change termination, [1, 2]): Let $\mathbb{G}$ be a set of size-change graphs. We say that $\mathbb{G}$ satisfies the size-change termination (SCT) condition if every infinite $\mathbb{G}$ -multipath has infinite descent.

3 Circular proof systems and the global trace condition

The *global trace condition* (GTC) is a criterion used to verify the validity of proofs in *circular* and *infinite proof systems*. In circular systems, proofs are represented as finite graphs with cycles, while in infinite systems, proofs take the form of infinite trees. Although the GTC admits different interpretations depending

on the underlying proof system, a general abstraction can be formulated. In this work, we adopt a system-independent notion of the GTC, as presented by Brotherston [4] and further developed with a categorical framework by Afshari and Wehr [5]. In the case of latter, we consider the $\mathbb{B}$ -activated trace category $\mathcal{T}_{\mathbb{B}}$, where $\mathbb{B} = \{\top, \perp\}$ is the binary boolean algebra. Although their framework provides a more general setting for describing trace conditions in circular proof systems, we will primarily rely on Brotherston's formulation to introduce the key concepts, for the sake of simplicity. Throughout this section, we focus exclusively on circular proof systems and do not consider infinite proof systems.

3.1 Arbitrary circular proof systems

An arbitrary *circular proof system* CS^ω is given by a set of well-formed sequents Seqs , and a set of rules Rules , where each rule $R \in \text{Rules}$ is defined as:

$$\frac{S_1 \dots S_n}{S} (R)$$

where $n \in \mathbb{N}$ and each S_i and S belong to Seqs . To formally represent the graph structure of a derivation in CS^ω , we define the notions of a *derivation graph* and a CS^ω *pre-proof*.

Definition 3.1. (Derivation graph, [4]): A CS^ω derivation graph, is a tuple $\mathcal{D} = (V, s, r, p)$ where:

- V is a set of vertices
- $s : V \rightarrow \text{Seqs}$ is total function that labels vertices with sequents in CS^ω
- $r : V \rightarrow \text{Rules}$ is partial function that labels vertices with inference rules in S
- $p : \mathbb{N} \times V \rightarrow V$ is a partial function that maps a number n and a vertex v , with a vertex v' , such that v' is labeled by the n^{th} premise of the rule $r(v)$ over the sequent $s(v)$.
- For every $v \in V$, the following inference is an instance of the rule $r(v)$ in S , with m premises:

$$\frac{\{\vdash s(p(i, v))\}_{1 \leq i \leq m}}{\vdash s(v)} (r(v))$$

A derivation graph $\mathcal{D}$ is called a *derivation tree* if its directed graph is a tree. Moreover, $\mathcal{D}$ is *finite* (resp. *infinite*) if the cardinality of V is finite (resp. infinite). We say that S is the *endsequent* of a derivation tree $\mathcal{D}$, if the root node of the tree is labeled by S . A vertex $B \in V$ such that $r(B)$ is undefined. The set of bud nodes of a derivation graph $\mathcal{D}$, is written as $\text{Bud}(\mathcal{D})$.

Definition 3.2. (Circular pre-proof, [4]): A CS^ω pre-proof of a sequent S , is represented by a tuple $\mathcal{P} = (\mathcal{D}, \mathcal{R})$, where $\mathcal{D}$, is a CS^ω finite derivation tree with endsequent S , and $\mathcal{R} : V \rightarrow V$ is a partial function, that maps each one of the bud nodes in $\text{Bud}(\mathcal{D})$ to an internal node in the derivation tree, called a *companion*, satisfying that if $B \in \text{Bud}(\mathcal{D})$ and $\mathcal{R}(B) = C$, then $r(C)$ is defined and $s(B) = s(C)$. We define the set of bud nodes of $\mathcal{D}$, as $\text{Com}(\mathcal{D})$.

The combination of both the derivation tree $\mathcal{D}$ and the map $\mathcal{R}$ of a pre-proof $\mathcal{P}$, form a graph that can contain cycles. We call this graph $\mathcal{G}_{\mathcal{P}}$, and it is formally defined as:

Definition 3.3. (Pre-proof graph, [4]): Let $\mathcal{P} = (\mathcal{D} = (V, s, r, p), \mathcal{R})$ be a CS^ω pre-proof. The graph of $\mathcal{P}$, written as $\mathcal{G}_{\mathcal{P}}$, is defined as $\mathcal{G}_{\mathcal{P}} = (V', s, r, p')$ where:

- $V' = V \setminus \text{Bud}(\mathcal{D})$
- p' is defined as:

$$p'_j(v) \simeq \begin{cases} \mathcal{R}(p_j(v)) & \text{if } p_j(v) \in \text{Bud}(\mathcal{D}) \\ p_j(v) & \text{otherwise} \end{cases}$$

for some $j \in \mathbb{N}$. We write $\simeq$ to say that $p'_j(v)$ is defined iff $\mathcal{R}(p_j(v))$ or $p_j(v)$ are defined, depending on the case.

A pre-proof graph $\mathcal{G}_{\mathcal{P}}$ can be seen as a directed graph (V', E) where the set of edges is defined as $E = \{(v, p'_j(v)) \mid v \in V \text{ and } p'_j(v) \text{ is defined}\}$. A *path* in $\mathcal{G}_{\mathcal{P}}$ is a finite or infinite sequence of vertices $v_0 j_0 v_1 j_1 v_2 j_2 \dots$ such that for all $i \geq 0$, $v_i \in V$, $j_i \in \mathbb{N}$ and $v_{i+1} = p'_{j_i}(v_i)$. Sometimes, we will also write only $v_0 v_1 v_2 \dots$ to specify a path without the successor indexes.

Example 3.1. We will use an example of a circular pre-proof in the CLKID^ω proof system, taken from [4], to illustrate the previous definitions. In this system, inductive predicates are allowed, and we assume that the domain of the variables is $\mathbb{N}$ (or any other standard model of arithmetic). Each natural number is represented inductively, either as 0 or using the successor function s . The rules for the inductive predicates N , P , and Q are defined in Figure 3. Here, P and Q are mutually dependent inductive predicates, while N corresponds to the standard inductive definition of the natural numbers.

$$\begin{array}{c}
\frac{}{\Gamma \vdash P0, \Delta} \text{ (PR}_1\text{)} \qquad \frac{\Gamma \vdash Px, \Delta \quad \Gamma \vdash Q(x, sx), \Delta}{\Gamma \vdash Psx, \Delta} \text{ (PR}_2\text{)} \\
\frac{}{\Gamma \vdash Q(x, 0), \Delta} \text{ (QR}_1\text{)} \qquad \frac{\Gamma \vdash Q(x, y), \Delta \quad \Gamma \vdash Px, \Delta}{\Gamma \vdash Q(x, sy), \Delta} \text{ (QR}_2\text{)} \\
\frac{\Gamma, x = 0 \vdash \Delta \quad \Gamma, x = sz, Nz \vdash \Delta}{\Gamma, Nx \vdash \Delta} \text{ (Case } N\text{)}
\end{array}$$

Figure 3: Rules in CLKID^ω , for the unary predicates N, P and the binary predicate Q

Figure 4 presents an example of a circular pre-proof in the CLKID^ω proof system, which uses the rules defined for the inductive predicates P , Q , and N , along with standard structural rules such as contraction and weakening, as well as rules associated with first-order proof systems, such as equality substitution and variable renaming. The derivation graph corresponds to the tree structure shown in Figure 4, where the bud nodes are the leaves that have no rule associated with them, e.g. the leaves labeled with the sequent $Nx, Ny \vdash Q(x, y)$. The function $\mathcal{R}$ is represented by labeling each bud and its corresponding companion node with the same symbol, for example, the buds labeled with $(\dagger_1)$ and $(\dagger_2)$ map to the companion labeled with $(\dagger)$, and similarly for the vertices labeled with $(*)$. The graph $\mathcal{G}_{\mathcal{P}}$ is then obtained by adding the edges induced by $\mathcal{R}$ and deleting the bud vertices from Figure 4.

$$\begin{array}{c}
\frac{Nx \vdash Px (*)}{Nz \vdash Pz} \text{ (Subst)} \quad \frac{Ny, Nx \vdash Q(x, y) (\dagger_2)}{Nsz, Nz \vdash Q(z, sz)} \text{ (Subst)} \\
\frac{\frac{}{\vdash P0} \text{ (PR}_1\text{)}}{x = 0 \vdash Px} \text{ (=L)} \quad \frac{\frac{Nsz, Nz \vdash Psz}{Nx, x = sz, Nz \vdash Px} \text{ (=L)}}{Nx, Nx \vdash Px} \text{ (Case } N\text{)} \\
\frac{Nx, Ny \vdash Q(x, y) (\dagger_1)}{Nx, Nz \vdash Q(x, z)} \text{ (Subst)} \quad \frac{Nx, Nx \vdash Px}{Nx \vdash Px (*)} \text{ (Contr L)} \\
\frac{Nx \vdash Q(x, 0)}{Nx, y = 0 \vdash Q(x, y)} \text{ (QR}_1\text{)} \quad \frac{Nx, Nz \vdash Q(x, sz)}{Nx, y = sz, Nz \vdash Q(x, y)} \text{ (QR}_2\text{)} \\
\frac{Nx, y = 0 \vdash Q(x, y)}{Nx, Ny \vdash Q(x, y) (\dagger)} \text{ (=L)} \quad \frac{Nx, y = sz, Nz \vdash Q(x, y)}{Nx, Ny \vdash Q(x, y) (\dagger)} \text{ (Case } N\text{)}
\end{array}$$

Figure 4: Example of a circular proof in CLKID^ω of the sequent $Nx, Ny \vdash Q(x, y)$. The double horizontal lines in some of the rules represent a hidden weakening rule.

3.2 Generalized GTC

In most circular proof systems, the validity or soundness of circular pre-proofs is expressed via the global trace condition (GTC). This condition can be generally stated as follows: “A CS^ω pre-proof $\mathcal{P}$ satisfies the global trace condition, in which case we call it a CS^ω proof, if and only if every infinite path in $\mathcal{G}_{\mathcal{P}}$ has a

trace that *infinitely progresses*.” However, both the notion of a “trace” and what it means to “infinitely progressing” are defined differently depending on the circular proof system under consideration. Moreover, the *semantic interpretation* of the GTC, which provides the soundness criterion for each system, also varies based on the specific characteristics of the circular proof system. For these reasons, we will devote the remainder of this section to presenting a generalized version of these concepts.

First, we introduce an abstract notion of semantics for a circular proof system. Given a circular proof system CS^ω , we define $\mathbb{I}$ as a set of *interpretations*, and write $I \models S$ to mean that the sequent S is satisfied by the interpretation $I \in \mathbb{I}$. Moreover, a sequent S is said to be *valid* if it is satisfied by every interpretation $I \in \mathbb{I}$. The definition of each individual interpretation I depends on the characteristics of the proof system. For example, a circular proof system involving predicates and first-order quantification, like CLKID^ω [4], may be interpreted by a first-order structure $\mathcal{M}$ together with an environment ρ , whereas other systems such as the μ -calculus [CITATIONS] use only an environment for the interpretation of the variables of a sequent over a particular domain. Therefore, to define the general notion of a trace, we introduce a *trace function*:

Definition 3.4. (Trace function & trace, [4]): Let $\mathcal{T}$ be a set, CS^ω a circular proof system, and $TVal \subseteq \mathcal{T} \times \text{Seqs}$ a relation satisfying that for all $S \in \text{Seqs}$, there are finitely many $\tau \in \mathcal{T}$ such that $TVal(\tau, S)$, and we define $TVal_S = \{\tau \mid TVal(\tau, S)\}$. Moreover, let $TPair : (\mathcal{T} \times \mathcal{T}) \rightarrow (\text{Seqs} \times \text{Rules} \times \text{Seqs}) \rightarrow \{0, 1, 2\}$ be a computable function satisfying:

$$\forall \tau, \tau', s, r, s'. \neg TVal(\tau, s) \vee \neg TVal(\tau', s') \Rightarrow TPair(\tau, \tau')(s, r, s') = 0$$

Also, suppose there exists a function $\sigma : \mathcal{T} \times \mathbb{I} \rightarrow O$, where O is some initial segment of the ordinals satisfying that for any CS^ω pre-proof $\mathcal{P}$ with a graph $\mathcal{G}_\mathcal{P} = (V, s, r, p)$ then:

$$\begin{aligned} I \not\models s(v) &\Rightarrow \exists I', v', j. v' = p_j(v) \\ &\text{and } I' \not\models s(v') \\ &\text{and } TPair(\tau, \tau')(s(v), r(v), s(v')) = 1 \Rightarrow \sigma(\tau', I') \leq \sigma(\tau, I) \\ &\text{and } TPair(\tau, \tau')(s(v), r(v), s(v')) = 2 \Rightarrow \sigma(\tau', I') < \sigma(\tau, I) \end{aligned} \quad (1)$$

$TVal$ is called a *trace value relation* and $TPair$ is called a *trace pair function* for the proof system, and σ is the *ordinal trace function* associated with $TPair$ and $TVal$. The pair (τ, τ') is said to be a *valid trace pair* on (v, v') if $TPair(\tau, \tau')(s(v), r(v), s(v')) \neq 0$, a *weakly-progressing trace pair* on (v, v') if $TPair(\tau, \tau')(s(v), r(v), s(v')) = 1$, and a *progressing trace pair* on (v, v') if $TPair(\tau, \tau')(s(v), r(v), s(v')) = 2$. Moreover, a sequence (τ_i) of elements in $\mathcal{T}$ is a *trace* on a path (v_i) , if for all i , (τ_i, τ_{i+1}) is a valid trace pair on (v_i, v_{i+1}) . A trace (τ_i) is said to *progress* at j if (τ_j, τ_{j+1}) is a progressing trace pair, and is said to be *infinitely progressing* if there are infinitely many points at which it progresses.

Example 3.2. To understand the intuition behind the previous definitions, we use the Example 3.1. In the case of the CLKID^ω circular proof system, as stated earlier, an interpretation I for a sequent $\Gamma \vdash \Delta$ is given by the structure $\mathbb{N}$ together with an environment ρ for the values of the variables, written as $\mathbb{N} \models_\rho \Gamma \vdash \Delta$. The relation $TVal$ associates inductive predicates $P(\vec{x})$ with sequents, meaning that if $P(\vec{x})$ is a formula occurring in the sequent S , then $TVal(P(\vec{x}), S)$ holds. The function $TPair$ describes how inductive predicates transform after the application of each rule in the proof system. For example, consider the first instance of the rule (Case N), with v the root node, and v_l and v_r its left and right child nodes, respectively. Then, $TPair(Nx, Nx)(s(v), r(v), s(v_l)) = 1$, since the value of the variable x remains unchanged after the application of the rule, and $TPair(Ny, Nz)(s(v), r(v), s(v_r)) = 2$, since the value of the variable y is strictly decreased after the application of the rule.

The function σ is defined as follows: given an interpretation $I = (\mathbb{N}, \rho)$ and the inductive predicate Nx , we have $\sigma(Nx, I) = \rho(x)$. In the case where the inductive predicate involves n variables, we use the valuation $\rho(\vec{x}) = (\rho(x_1), \dots, \rho(x_n))$ and apply the lexicographical order. Then, the trace $\tau = (Ny, Nz, Nz, Nz, Ny)^\omega$ is infinitely progressing, since the pair $TPair(Ny, Nz)(s(v), r(v), s(v_r))$ is progressing. Moreover, this trace can be used to validate all the infinite paths contained in the cycle from $(\dagger)$ to $(\dagger_1)$.

Having defined the notion of a trace, we can now formally define the generalized version of the GTC. With this notion of the GTC, Brotherston proved that the generalized GTC implies the soundness of proofs

in infinitary proof systems, by relying in the infinite descent method and the condition (1) of the Definition 3.4. Moreover, his proof also implies the soundness of CS^ω proofs, since circular proofs can be viewed as a subset of infinite proofs.

Definition 3.5. (Generalized global trace condition, [4]): An CS^ω pre-proof $\mathcal{P}$ is said to be an CS^ω proof if, for every infinite path in $\mathcal{G}_\mathcal{P}$ there is an infinitely progressing trace following some tail of the path.

Proposition 3.1. (Soundness of generalized circular proofs, [4]): If there is a CS^ω proof of the sequent S , then S is valid.

Example 3.3. We do not include the full proof of Proposition 3.1, as it can be found in the original work by Brotherston [4]. Instead, we provide an informal explanation using Example 3.1. The goal is to justify that in the circular proof shown in Figure 4, the sequent $Nx, Ny \vdash Q(x, y)$ is valid, i.e. that for any environment ρ , $\mathbb{N} \models_\rho Nx, Ny \vdash Q(x, y)$. For those branches of the proof ending in axiomatic rules, validity can be established directly by induction on the rules themselves. However, this approach does not work for branches whose leaves are bud nodes, as these branches are not well-founded. In such cases, validity is established using the method of infinite descent.

Suppose, for contradiction, that there exists an environment ρ such that $\mathbb{N} \not\models_\rho Nx, Ny \vdash Q(x, y)$. Consider the branch corresponding to the bud node labeled $(\dagger_1)$. Since we know this path has an infinitely progressing trace, namely $\tau = (Ny, Nz, Nz, Nz, Ny)^\omega$, the interpretation of this trace under ρ would yield an infinite descending chain of natural numbers. But this is impossible in $\mathbb{N}$, and thus our assumption must be false. Therefore, $\mathbb{N} \models_\rho Nx, Ny \vdash Q(x, y)$. Since each infinite path in $\mathcal{G}_\mathcal{P}$ represents a derivation, we must find an infinitely progressing trace for each one in order to establish the validity of the entire circular proof. It is easy to see, that for the pre-proof in Figure 4, every infinite path has a infinitely progressing trace, hence it is a CLKID^ω proof.

4 Equivalence between GTC and SCT

It has long been part of the folklore that the GTC and the SCT principle are equivalent. However, to the best of the authors' knowledge, this equivalence has only been formally established in the work of Ikebuchi [2], specifically for the CLKID^ω circular proof system. For this reason, the following section is based on her results, with adaptations to the setting of the generalized GTC introduced earlier. The main objective of this section is to shed light on the formal correspondence between the generalized global trace condition and size-change termination.

The general idea is as follows: given a circular pre-proof $\mathcal{P} = (\mathcal{D}, \mathcal{R})$ with a set of companion nodes $\text{Com}(\mathcal{D}) = \{C_1, \dots, C_n\}$, we associate each companion node C_i with a function symbol in the size-change framework. The arity of this function is given by $|\text{TVals}_{s(C_i)}|$, that is, the number of values associated with the sequent $s(C_i)$. With this in mind, we are now ready to define the size-change graphs associated with a circular pre-proof.

Definition 4.1. (Size-change graphs of a circular pre-proof, [2]): Let $\mathcal{P} = (\mathcal{D}, \mathcal{R})$ be a circular pre-proof in CS^ω , with $\text{Bud}(\mathcal{D}) = \{B_1, \dots, B_n\}$ and $\text{Com}(\mathcal{D}) = \{C_1, \dots, C_m\}$. Now consider any $B_i \in \text{Bud}(\mathcal{D})$ and $C_j \in \text{Com}(\mathcal{D})$, such that C_j is an ancestor of B_i , i.e. there is a path from C_j to B_i in $\mathcal{D}$. Then, we define the size-change graph $G_{C_j, \mathcal{R}(B_i)} = (\text{TVals}_{s(C_j)}, \text{TVals}_{s(\mathcal{R}(B_i))}, E)$, where E is defined as:

- $G_{C_j, \mathcal{R}(B_i)}$ has an arc $\tau_k^{C_j} \xrightarrow{\downarrow} \tau_l^{\mathcal{R}(B_i)}$ iff there is a progressing trace from $\tau_k \in \text{TVals}_{s(C_j)}$, to $\tau_l \in \text{TVals}_{s(\mathcal{R}(B_i))}$.
- $G_{C_j, \mathcal{R}(B_i)}$ has an arc $\tau_k^{C_j} \xrightarrow{\uparrow} \tau_l^{\mathcal{R}(B_i)}$ iff there is a non-progressing trace from $\tau_k \in \text{TVals}_{s(C_j)}$, to $\tau_l \in \text{TVals}_{s(\mathcal{R}(B_i))}$.

We define $\mathbb{G}_\mathcal{P}$, as the set of all such size-change graphs in $\mathcal{P}$.

The equivalence between the GTC and the SCT principle was formally established by Ikebuchi [2] for the CLKID^ω circular proof system. We now restate this result in a generalized setting, applying it to abstract circular proof systems.

Proposition 4.1. (GTC and SCT equivalence, [2]): For any CS^ω proof $\mathcal{P}$, $\mathbb{G}_{\mathcal{P}}$ satisfies the SCT condition. Conversely, for any CS^ω pre-proof $\mathcal{P}$, if $\mathbb{G}_{\mathcal{P}}$ satisfies the SCT condition, then $\mathcal{P}$ is a proof.

This proposition can be proved using the original proof strategy given by Ikebuchi in [2]. While her result was presented in the specific context of CLKID^ω , the underlying reasoning is sufficiently general to apply to arbitrary circular proof systems.

Example 4.1. (USE IKEBUCHI'S OR ANOTHER EXAMPLE, TO GIVE INTUITION BEHIND THE PROPOSITION)

5 Abstract SCT/GTC and the progress domain condition:

Given the equivalence between SCT and GTC established in Proposition 4.1, it becomes evident that both conditions share two key characteristics: (1) both can be formulated using size-change graphs, and (2) the composition of these graphs relies on an underlying structure. In the case of SCT, this structure arises from the control flow between function calls. For GTC, it is determined by the position of the companion nodes in the graph structure of circular pre-proofs. Motivated by these commonalities, we now introduce a unified abstraction that captures both SCT and GTC within a single formal definition.

5.1 Bi-colored relation transition graphs:

Building on observation (1), we begin by abstracting size-change graphs as *bi-colored relations* over a finite set of variables. which will allow for a uniform treatment of both SCT and GTC.

Definition 5.1. (Bi-colored relations and pair composition): Let $X = \{x_1, \dots, x_n\}$ be a finite set of variables. A bi-colored relation over X , is a pair (S, P) such that $S, P \subseteq X \times X$ where $S_i \cap P_i = \emptyset$. For two variables $x, y \in X$, If xSy (resp. xPy) we say that x *weakly progresses* (resp. *progresses*) to y in (S, P) . Moreover, we define the composition of two pairs (S, P) and (S', P') as:

$$(S, P)(S', P') = (S'' \setminus P'', P'')$$

where S'' and P'' are defined as:

$$S'' = S \circ S' \quad P'' = (S \circ P') \cup (P \circ S') \cup (P \circ P')$$

To capture observation (2), we introduce a transition system whose edges are labeled by bi-colored relations, abstracting both the control flow between function calls in SCT and the proof structure in circular pre-proofs for GTC. This general notion provides a common foundation for the unified treatment of SCT and GTC.

Definition 5.2. (Bi-colored relation transition graph): Let $\mathcal{B} = \{(S_1, P_1), \dots, (S_n, P_n)\}$ be a set of bi-colored relations. We define a *bi-colored relation transition graph*, as a labeled transition system represented by the tuple $\mathcal{T} = (V, \mathcal{B}, E)$ where: V is a finite non-empty set of vertices and $E \subseteq V \times \mathcal{B} \times V$ is a set of directed edges labeled by bi-colored relations in $\mathcal{B}$.

Definition 5.3. (Composable)

Example 5.1. [Include here an example (THE SAME AS FOR SECTION 4) of how the three defintions SCT $\rightarrow$ GTC $\rightarrow$ transition graph interact][Make basically a triangle where each corner is each representation and each edge is a biconditional $\rightarrow$]

5.2 sdas

References

- [1] C. S. Lee, N. D. Jones, and A. M. Ben-Amram, “The size-change principle for program termination,” in *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '01, (New York, NY, USA), p. 81–92, Association for Computing Machinery, 2001.

- [2] M. Ikebuchi, “Cyclic proofs and size-change termination,” *Theoretical Computer Science*, vol. 1045, p. 115273, 2025.
- [3] C. S. Lee, “Ranking functions for size-change termination,” *ACM Trans. Program. Lang. Syst.*, vol. 31, Apr. 2009.
- [4] J. Brotherston, “Sequent calculus proof systems for inductive definitions,” 2006.
- [5] B. Afshari and D. Wehr, “Abstract cyclic proofs,” *Mathematical Structures in Computer Science*, vol. 34, no. 7, p. 552–577, 2024.